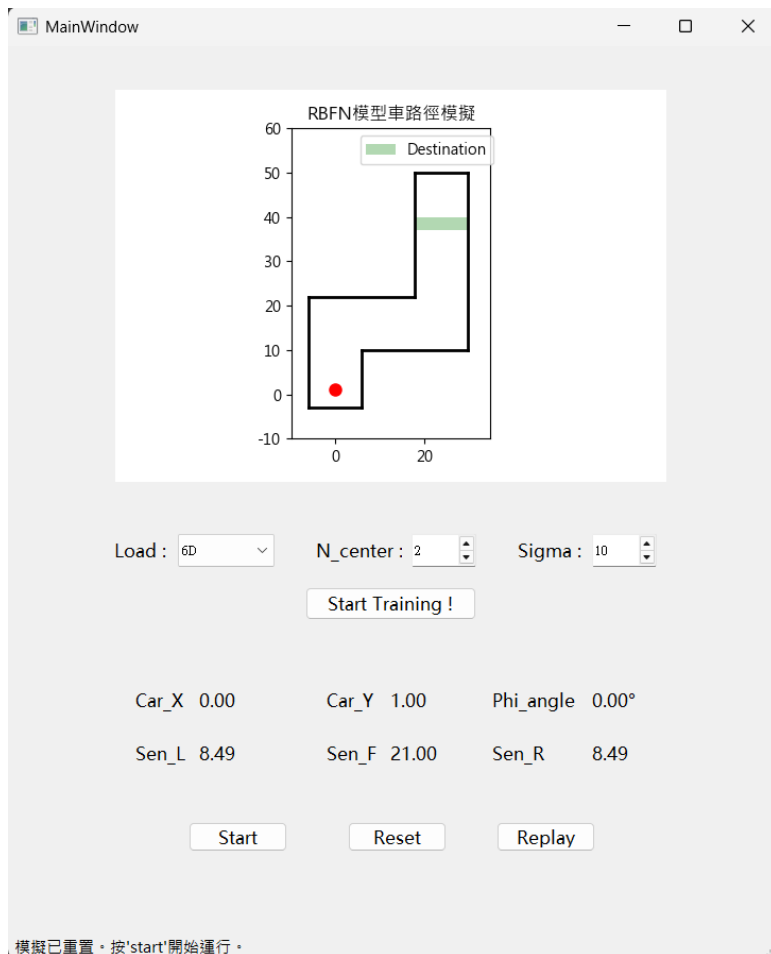


類神經網路作業二 — 自駕車

A. GUI 功能說明



訓練與模型設定區：

Load：選擇訓練資料(6D:[x, y, df, dr, dl] / 4D:[df, dr, dl])

N_center：設定 RBFN 隱藏層的中心點數量

Sigma：設定 RBFN 網路高斯徑向基函數的寬度參數

Start Training：根據上述設定載入對應數據，訓練 RBFN 模型；訓練完成後將顯示 RMSE 於視窗最底部

數值和控制面板：

Car_X / Car_Y：自駕車中心的即時座標

Phi_angle：模型在當下輸出的建議方向盤角度

Sen_L / Sen_F / Sen_R：自駕車即時偵測到左、前、右方到牆壁的距離

Start：啟動即時控制模擬，根據感測器讀數，通過訓練好的 RBFN 模型輸出方向盤角度，調整自駕車的運行方向

Reset：停止模擬、重置車輛位置並清除軌跡

Replay：讀取前一次成功模擬儲存的檔案，並重現該路徑

B. 程式碼簡介

1. train_utils.py : 負責模型訓練

load_and_parse_data : 根據 user 選擇的模式(4D 或 6D)，讀取檔案並將原始數據分割為 feature X 和 output Y。

```
# 根據模式載入並解析訓練數據
def load_and_parse_data(is_6d_mode:bool)-> Tuple[Optional[np.ndarray], Optional[np.ndarray]]:
    # is_6d_mode == true -> 6D (X, Y, D_f, D_r, D_l)
    # is_4d_mode == false -> 4D (D_f, D_r, D_l)
    filename = FILE_6D if is_6d_mode else FILE_4D

    try:
        df = pd.read_csv(filename, header = None, sep = "\s+")
    except FileNotFoundError:
        print(f"錯誤：找不到檔案 {filename}，請檢查路徑。")
        return None, None

    # 刪除可能因為讀取空格導致的多餘 NaN 欄位
    df = df.dropna(axis=1, how='all')

    if df.empty or df.shape[1] < 2:
        print(f"錯誤：檔案 {filename} 讀取後數據不足。")
        return None, None

    # X 是前 n-1 欄，Y 是最後一欄
    X = df.iloc[:, :-1].values
    Y = df.iloc[:, -1].values.reshape(-1, 1)

    return X, Y
```

RBFN.train : 實現 RBFN 模型的 2 階段訓練

1. 使用 K-Means 演算法確定隱藏層中心點
2. 使用 Pseudo inverse 求解輸出權重 W。

```
# RBFN訓練步驟：K-Means找中心點，虛反矩陣找權重
def train(self, X_train: np.ndarray, Y_train: np.ndarray):

    if X_train is None or Y_train is None:
        raise ValueError("訓練數據無效，無法啟動訓練。")

    # 步驟 1: K-Means 確定中心點
    kmeans = KMeans(n_clusters=self.n_centers, random_state=0, n_init='auto')
    kmeans.fit(X_train)
    self.centers = kmeans.cluster_centers_

    # 步驟 2: 計算隱藏層輸出矩陣 H
    H = self._calculate_hidden_layer_output(X_train)

    # 步驟 3: 求解輸出權重 W (虛反矩陣)
    H_pseudo_inverse = np.linalg.pinv(H)
    self.weights = H_pseudo_inverse @ Y_train
```

RBFN.predict：接收模擬環境的即時狀態 X ，透過訓練好的中心點和權重計算 RBF 輸出，最終返回建議的方向盤角度。

```
# 預測最佳方向盤角度phi
def predict(self, X_input: List[float]) -> float:
    if self.weights is None or self.centers is None:
        # 確保模型已訓練
        raise ValueError("模型尚未訓練。")

    # 將輸入列表轉換為 NumPy 陣列 (1, n_features)
    X_input_array = np.array(X_input).reshape(1, -1)

    # 計算隱藏層輸出 H
    H = self._calculate_hidden_layer_output(X_input_array)

    # 輸出層輸出 Y_pred = H @ W
    Y_pred = H @ self.weights

    # 返回角度的單一數值
    return Y_pred[0, 0]
```

2. gui_run.py：負責運行與展示

train_model：接收 GUI 中的 hyperparameter(N_center, sigma)和模式選擇，呼叫 train_utils.RBFN.train()函數，並將訓練結果化為 self.controller，以實現動態訓練的能力。

```
def train_model(self):
    self.timer.stop()
    self.set_ui_interaction_enabled(False)
    self.train_button.setEnabled(False)

    # 1. 讀取用戶設定和超參數
    try:
        # 從 GUI 元件中讀取數據
        n_centers = int(self.spinBox_nc.text())
        sigma = float(self.spinBox_sigma.text())

        # 判斷數據模式
        data_mode = self.data_selector.currentText()
        is_6d_mode = (data_mode == "6D")

    except ValueError:
        self.statusBar().showMessage("訓練錯誤：N_CENTERS 和 SIGMA 必須為有效數字。", 5000)
        return

    if n_centers <= 0 or sigma <= 0:
        self.statusBar().showMessage("訓練錯誤：中心點和 Sigma 必須大於零。", 5000)
        return

    self.statusBar().showMessage(f"開始訓練 RBFN 模型 ({data_mode} 模式, Centers={n_centers}, Sigma={sigma})...", 0)

    # 2. 數據載入
    X_train, Y_train = load_and_parse_data(is_6d_mode)

    if X_train is None:
        self.statusBar().showMessage("訓練失敗：數據載入錯誤。", 5000)
        return

    # 3. 執行訓練
    try:
        rbf_model = RBFN(n_centers=n_centers, sigma=sigma)
        rbf_model.train(X_train, Y_train) # 這是阻塞操作，可能會讓 GUI 短暫卡住

        # 4. 評估與更新控制器
        rmse = rbf_model.evaluate(X_train, Y_train)
        self.controller = rbf_model

        self.statusBar().showMessage(f"訓練成功！RMSE: {rmse:.4f}度。模型已準備就緒。", 0)

    except Exception as e:
        self.statusBar().showMessage(f"訓練失敗：發生內部錯誤 - {e}", 8000)
        self.controller = None

    finally:
        self.set_ui_interaction_enabled(True)
        self.train_button.setEnabled(True)

    self.is_6d_trained = is_6d_mode
```

`run_next_step`：模擬運行的入口，由 `QTimer` 定時驅動
負責判斷當前模式(simulation / playback)，並呼叫對應的邏輯函數
(`_run_simulation_step` 或 `_run_playback_step`)。

```
# QTimer 觸發的單一入口，根據模式執行對應邏輯。
def run_next_step(self):
    if self.is_playback_mode:
        self._run_playback_step()
    else:
        self._run_simulation_step()
```

`_run_simulation_step`：實現即時控制的功能
獲取感測器狀態 -> 呼叫 `self.controller.predict()` -> 即時記錄(X, Y, phi, dists)

```
# 即時控制模式：模型預測並推進運動學。
def _run_simulation_step(self):
    # 1. 檢查結束條件
    self.playground._checkDoneIntersects()
    if self.playground.done:
        self.timer.stop()
        self.check_end_status(is_playback=False) # 區分模式
        return

    # 2. 獲取數據
    current_x = self.playground.car.xpos
    current_y = self.playground.car.ypos
    sensor_dists = self.playground.state # [D_f, D_r, D_l]
    full_input = [current_x, current_y] + sensor_dists

    # 3. 模型預測
    if self.is_6D_trained:
        input_features = full_input
    else:
        input_features = sensor_dists

    predicted_angle = self.controller.predict(input_features)

    # 4. 推進運動學
    self.playground.car.setWheelAngle(predicted_angle)
    self.playground.car.tick()

    # 5. 紀錄成功路徑 (僅在成功運行時)
    new_x = self.playground.car.xpos
    new_y = self.playground.car.ypos

    record_row = [
        new_x, new_y, sensor_dists[0], sensor_dists[1], sensor_dists[2], predicted_angle.item()
    ]
    self.path_record.append(record_row)

    # 6. 推進繪圖
    car_center = self.playground.car.getPosition("center")
    self._update_display(car_center, sensor_dists, predicted_angle.item())
```

`_run_playback_step`：實現讀取路徑記錄檔和碰撞偵測的功能

直接設定車輛位置和方向盤角度，並在每一步進行碰撞偵測直到路徑結束。

```
# 數據重放模式：讀取數據並設定位置
def _run_playback_step(self):
    if self.playback_step >= len(self.playback_data):
        self.timer.stop()
        self.is_playback_mode = False
        self.statusBar().showMessage("重放結束：路徑已完整重現。", 0)
        return

    # 1. 從檔案讀取數據 (格式: X, Y, Df, Dr, Dl, Phi)
    record_row = self.playback_data[self.playback_step]
    x_new, y_new, sensor_f, sensor_r, sensor_l, phi_angle = record_row

    # 2. 直接設定位置 (忽略模型和運動學)
    from simple_geometry import Point2D
    self.playground.car.setPosition(Point2D(x_new, y_new))
    self.playground.car.setWheelAngle(phi_angle)

    # 3. 碰撞檢查 (滿足作業要求)
    self.playground._checkDoneIntersects()
    if self.playground.done:
        self.timer.stop()
        self.is_playback_mode = False
        self.check_end_status(is_playback=True)
        return

    # 4. 推進繪圖 (使用檔案中的感測器數據)
    car_center = self.playground.car.getPosition("center")
    sensor_dists_playback = [sensor_f, sensor_r, sensor_l]

    # 修正 5：重放模式下的軌跡點添加
    self.path_x.append(x_new)
    self.path_y.append(y_new)

    self._update_display(car_center, sensor_dists_playback, phi_angle)
    self.playback_step += 1
```

`_update_display`：實現所有繪圖和標籤值的更新

包含感測器射線計算、軌跡繪製、即時數據(X, Y, phi, dists)的同步顯示。

```
# 統一繪圖和標籤更新(無論是模擬或是重放模式)
def _update_display(self, car_center, sensor_dists, phi_angle):
    # 1. 獲取當前位置
    x, y = car_center.x, car_center.y

    # 2. 確保即時模式下獲取最新的交點數據來繪製射線
    intersections = [self.playground.front_intersects, self.playground.right_intersects, self.playground.left_intersects]

    # 3. 更新軌跡線
    self.car_plot.set_data([x], [y])
    self.path_line.set_data(self.path_x, self.path_y)

    # 4. 獲取一次車頭角度，並定義相對角度(only once)
    car_angle_head = self.playground.car.angle # 車頭的絕對角度
    SENSOR_RELATIVE_ANGLES = [0, -45, 45]

    # 5. 更新感測器射線
    # 根據模式決定是否顯示射線
    visibility = not self.is_playback_mode

    for i, current_intersect_list in enumerate(intersections):
        start_x, start_y = x, y

        # 確保angle每次都從car_angle_head重新計算
        current_abs_angle = car_angle_head + SENSOR_RELATIVE_ANGLES[i]

        # 將角度轉換為弧度，強制使用math.radians確保正確性
        rad = m.radians(current_abs_angle)
```

```

# 判斷是否有交點
# 如果有交點，則使用交點位置
if current_intersect_list:
    end_x, end_y = current_intersect_list[0].x, current_intersect_list[0].y

# 處理暢通或是距離有效但intersects為空的情況
else:
    # 使用傳入的sensor_dists參數
    L = sensor_dists[i]

    if L == -1: # 假設該方向暢通
        # 為了快速實現，先假設不畫暢通的線，或將其長度設定為一個固定值
        L = 50

    # 計算終點(不論暢通或是有讀數，公式都一樣)
    end_x = x + L * m.cos(rad)
    end_y = y + L * m.sin(rad)

# 更新射線的數據
self.sensor_plots[i].set_data([start_x, end_x], [start_y, end_y])

# 設置射線可見性
self.sensor_plots[i].set_visible(visibility)

# 6. 更新標籤
self._update_labels(phi_angle, sensor_dists)

# 7. 快速重繪(只重繪發生變化的部分)
self.canvas.draw()

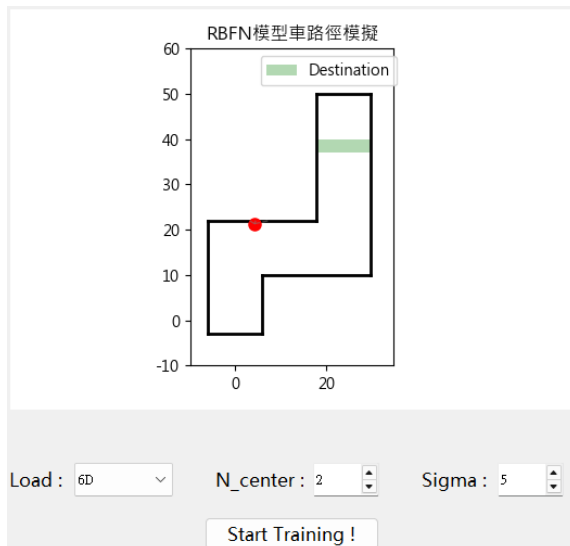
```

C. 實驗結果

模型訓練結果：

1. N_{center} , Σ 比較 (以 6D 為例)：

$(n_{center}, \sigma) = (2, 5)$,



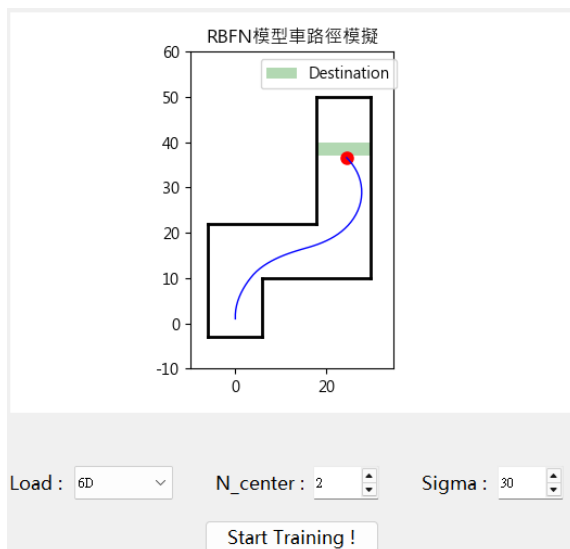
RMSE = 15.9748, steps = NaN (撞牆所以沒記錄)

$(n_{center}, \sigma) = (2, 10)$,



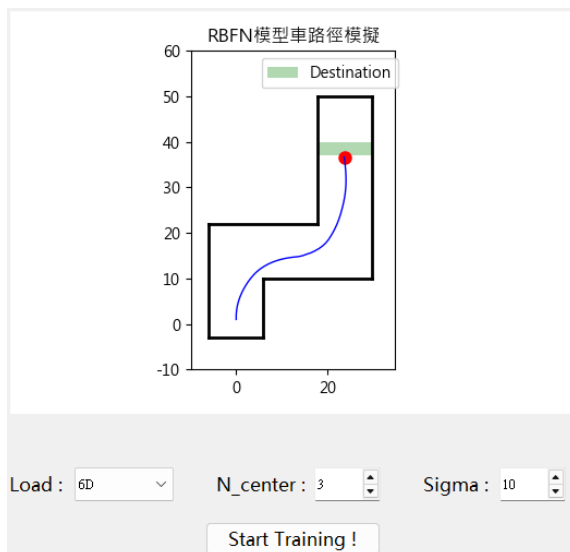
RMSE = 14.8641, steps = NaN (撞牆所以沒記錄)

$(n_center, \sigma) = (2, 30),$



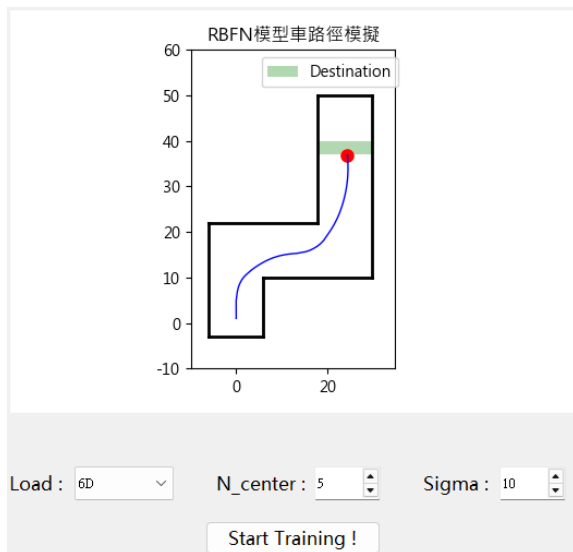
RMSE = 14.6025, steps = 53

$(n_center, \sigma) = (3, 10),$



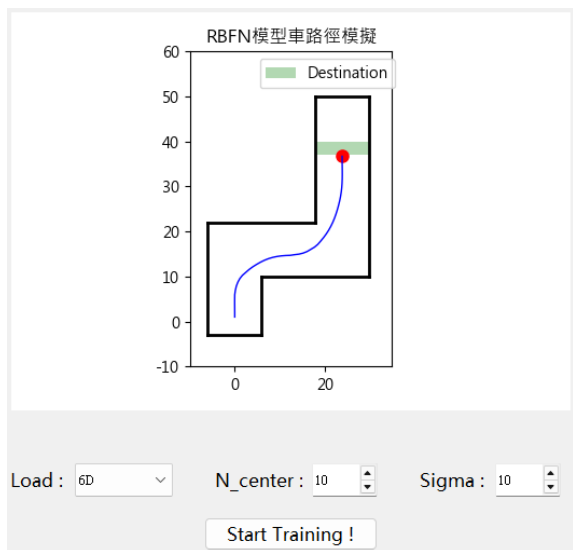
RMSE = 13.4352, steps = 50

$(n_center, \sigma) = (5, 10),$



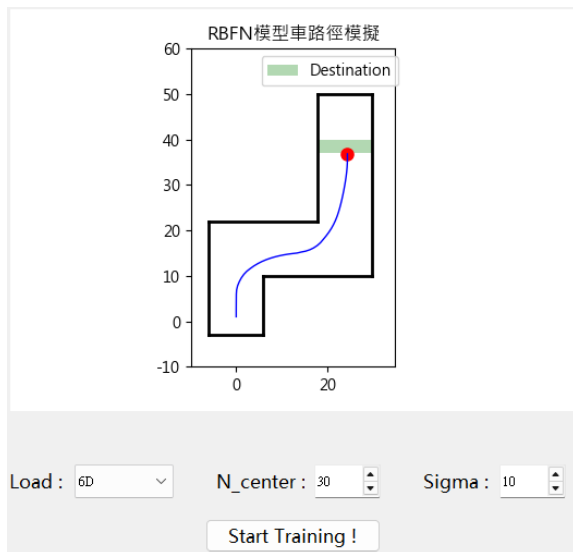
RMSE = 12.4441, steps = 51

$(n_center, \sigma) = (10, 10),$



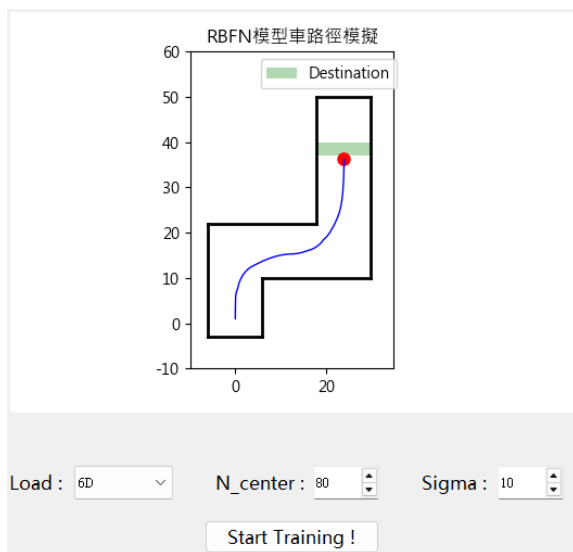
RMSE = 12.1204, steps = 51

$(n_center, \sigma) = (20, 10),$



RMSE = 11.2795, steps = 51

$(n_center, \sigma) = (80, 10),$



RMSE = 9.8693, steps = 51

2. 4D vs 6D (固定參數組為(2, 30), (3, 10))

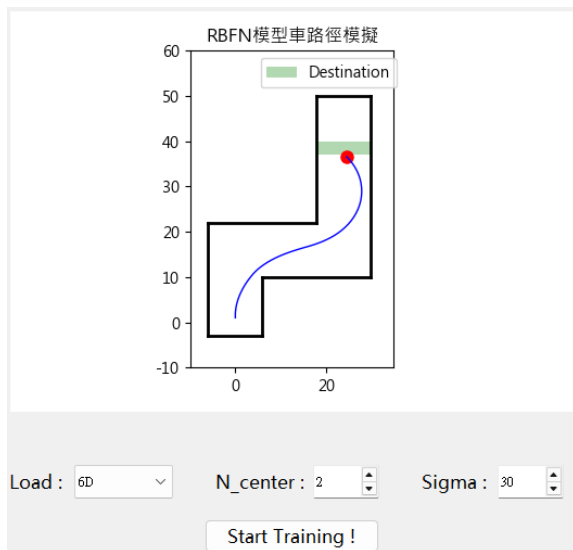
-> 因為此時 4D 和 6D 都尚未飽和)

(4D, 2, 30),



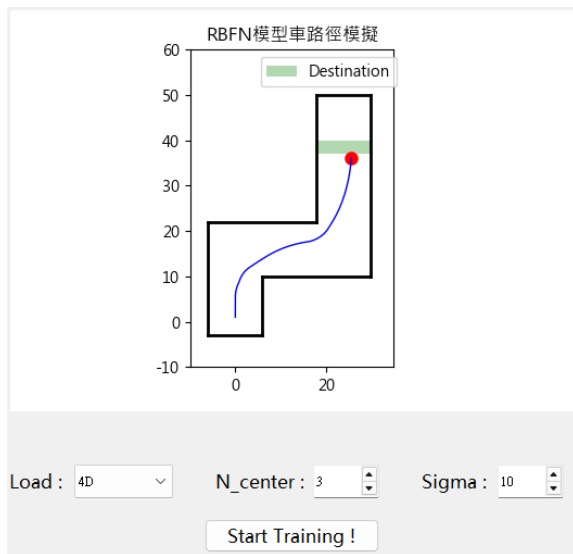
RMSE = 13.2557, steps = NaN

(6D, 2, 30),



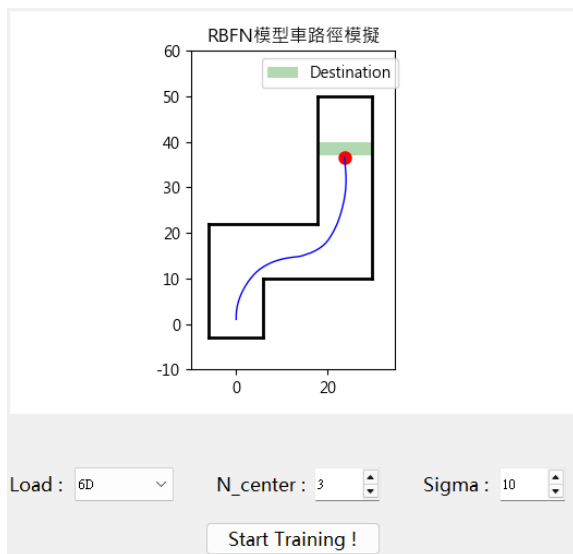
RMSE = 14.6025, steps = 53

(4D, 3, 10),



RMSE = 12.7989, steps = 50

(6D, 3, 10),



RMSE = 13.4352, steps = 50

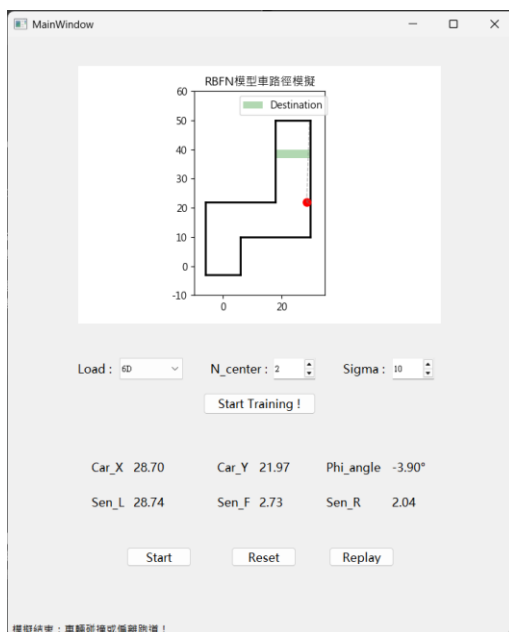
最佳模型運行軌跡：

參考”模型訓練結果”中展示的組合，我決定將最佳模型分成兩個層面討論 ->

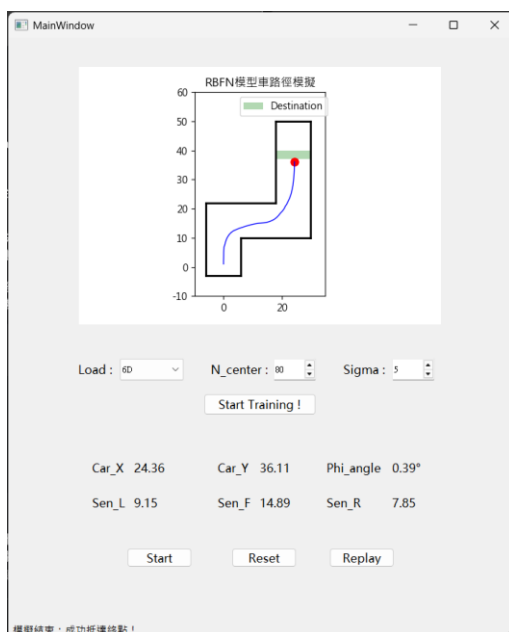
1. RMSE 表現： $(n_center, \sigma) = (80, 10)$, RMSE = 9.8693

2. 總步數表現： $(n_center, \sigma) = (3, 10)$, steps = 50

碰撞偵測驗證：經過數次實驗可觀察到，在觸發碰撞偵測時，感測器讀數通常停在 2.0 ~ 3.0 單位之間，這代表碰撞測試是有效的，由於感測器距離是從車輛的中心座標到牆壁的長度，而車輛半徑為 3 單位，因此感測器距離一旦靠近 3 單位，車體邊緣就會觸碰軌道。



重放功能驗證：



D. 分析

參數敏感度分析：

實驗結果顯示 RBFN 模型對 Sigma 的敏感度較低，但 N_center 的改變對模擬結果影響甚大。

N_center 的影響：

1. 當 N_center 極低時（例如 N_center = 2），模型無法擬合訓練數據，導致車輛在起點等數據稀疏區立即失控撞牆。
2. 當 N_center = 3 時達到了模型的最佳穩定點。此時剛好足夠捕捉到訓練數據中最核心的轉向趨勢，軌跡表現最為平滑且具備效率。
3. 當 N_center 增加到較高的值（例如 N_center = 20, 50, 80）時，儘管訓練 RMSE 略有下降，但在動態模擬中，車輛開始表現出不必要的抖動和微調。

Sigma 的影響：

實驗顯示，在 N_center 足夠的情況下，改變 Sigma（例如 5 到 10）對於模擬的最終穩定性和步數影響較小。這表明模型的性能主要由中心點的數量和分佈決定，而不是單個 RBF 節點的寬度。

4D / 6D 模式比較：

在本跑道上，4D 和 6D 模式的表現差異極小，顯示額外的全局位置信息對控制穩定性的邊際貢獻有限。

模型效率與選擇：

最佳選擇：(N_center = 3, Sigma = 10)，該模型以最短的 50 步完成模擬，這證明了在保證穩定性的前提下，這是幾何效率最高的路徑。